# Practical no. 18

**Aim - Continuous Integration using Jenkins**
Configure Jenkins to execute Selenium automation scripts automatically. Generate build reports and analyze test execution history.

**Tool:** Jenkins + Selenium WebDriver

**Theory –**

1. **Jenkins** is an open-source automation server used for Continuous Integration (CI) and automatically running software tests.
2. **Selenium WebDriver** is a browser automation tool used to automate and test web applications.
3. Jenkins can execute Selenium test scripts automatically using **Maven** as the build tool.
4. Test results and reports generated during execution can be used to monitor test execution and identify failures.
5. Jenkins maintains **build history**, allowing previous test executions and their results to be analyzed.

**Step 1: Install Jenkins**
First check whether Java is already installed.
Jenkins requires Java.
**Download Jenkins - D:\STQA**
You'll download a file similar to:
jenkins.msi

Double-click the downloaded .msi file.
The installer will guide you through several screens.
For a normal local installation, you can generally keep the default options.
When it asks about the **service logon credentials**, the default option is normally appropriate for a local lab setup.
When it asks for the **Jenkins port**, you will usually see: 8080
Keep **8080** unless another program is already using it.
So Jenkins will eventually be available at:

http://localhost:8080
Finish the installation.

**Step 2: Start Jenkins**
After installation, Jenkins should be installed as a Windows service.
You can check it:

1. Press Win + R.
2. Type:
3. services.msc
4. Press Enter.
5. Find **Jenkins**.
6. Its status should say **Running**.

If it isn't running, right-click **Jenkins → Start**.

**Step 3: Open Jenkins**

Open Chrome/Edge and enter: **http://localhost:8080**
You should get a screen saying something similar to:
Unlock Jenkins
This confirms that Jenkins is running on your computer.

**Step 4: Get the Initial Administrator Password**
Jenkins will ask you for an Administrator password during first-time setup.
On Windows, the password is normally stored in:

C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
Open that file using Notepad.

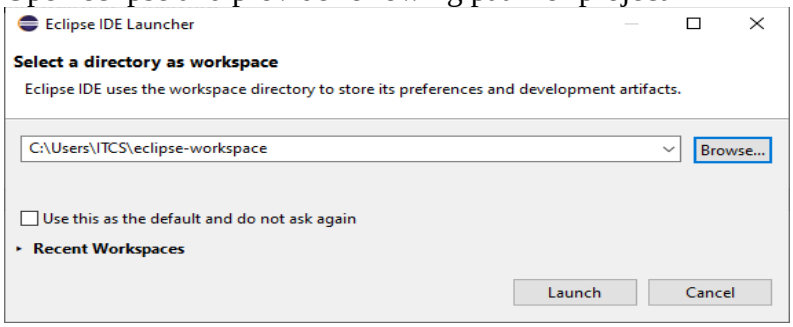
You can also open Command Prompt and run:
type C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
You'll see a long password, for example:
a1b2c3d4e5f6...

Copy it and paste it into the Jenkins browser page.

**Step 5: Complete Jenkins Setup**
Jenkins will then ask how you want to install plugins.
Choose:
**Install suggested plugins**
Wait for the installation to finish.
Then Jenkins will ask you to create your administrator account.
For example:
Username: admin
Password: ********
Create the account.

**Step 6: You should reach the Jenkins Dashboard**
You should now see something like:
Jenkins
Dashboard
Create a job
Build History
Manage Jenkins
Jenkins is installed!
Your local Jenkins address is:
**http://localhost:8080**

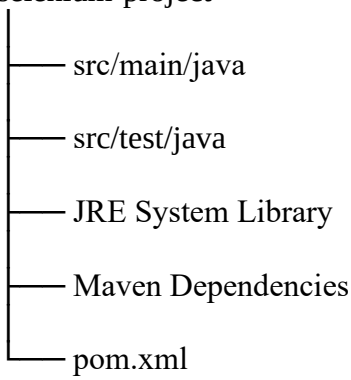
**Step 7: Create Maven Project**
Open eclipse and provide following path for project



From the Eclipse menu:
**File → New →** Maven Project
Tick on click a simple project
Click **Next**.

**Step 8: Enter Project Details**
Enter:
Group Id: com.example
Artifact Id: selenium-project
You can leave the other settings as default.
Click:
**Finish**
Eclipse will create your project.

You should see something similar to:
selenium-project




For our Selenium test, we'll use:
src/test/java

**Step 9: Open pom.xml**
In **Project Explorer**, double-click:
Replace its contents with:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.selenium</groupId>
    <artifactId>selenium-project</artifactId>
    <version>1.0-SNAPSHOT</version>
    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.35.0</version>
        </dependency>
```

```
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.11.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

Press: Ctrl + S
Then: Right-click project
→ Maven
→ Update Project
→ OK
Wait until Maven downloads the dependencies.

**Step 10 : Create Package**
In Eclipse: Right-click:
src/test/java
Select: **New → Package**
Enter: tests
Click **Finish**.
You now have:
src/test/java
└── tests

**Step 11 : Create Java Class**
Right-click: tests
Select: **New → Class**
Enter: LoginTest
Click **Finish**.
You now have:
src/test/java
└── tests
    └── LoginTest.java

Enter Selenium Code -
Open LoginTest.java.
Delete everything and paste:

```
package tests;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
public class LoginTest {
    private WebDriver driver;
    @BeforeMethod
```

```
    public void setUp() {
        // Selenium Manager automatically finds/downloads the
        // appropriate ChromeDriver. No driver path or version required.
        driver = new ChromeDriver();

        driver.manage().window().maximize();
    }
    @Test
    public void loginTest() {
        driver.get("https://www.google.com");
        System.out.println("Title: " + driver.getTitle());
    }
    @AfterMethod
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

Save: Ctrl + S

**Step 12: Create testng.xml**
Right-click your project:
selenium-project
→ New
→ File
Enter: testng.xml
Click **Finish**.

Paste:

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM
        "https://testng.org/testng-1.0.dtd">
<suite name="Selenium Test Suite">
    <test name="Chrome Tests">
        <classes>
            <class name="tests.LoginTest"/>
        </classes>
    </test>
</suite>
```

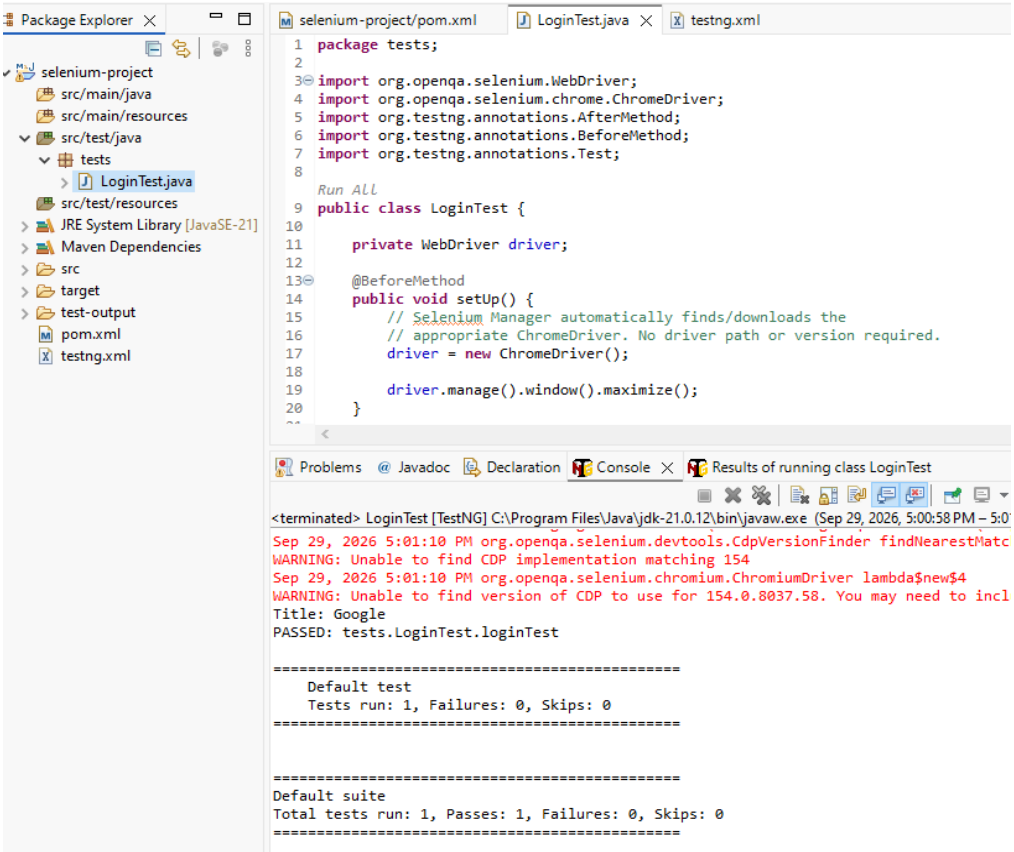
Save it.
Your project should now look like:
selenium-project

```
│
├── src/main/java
```

```
│
├── src/test/java
│   └── tests
│       └── LoginTest.java
│
├── Maven Dependencies
│
├── pom.xml
└── testng.xml
```

**Step 13: Run the Test**
Right-click:
LoginTest.java
Select: **Run As → TestNG Test**

Chrome should open.
The program will:
Start Chrome
↓
Open Google
↓
Get page title
↓
Verify title
↓
Close Chrome
At the bottom of Eclipse, you should see the **TestNG Results** view.



If getting - An internal error occurred during: "Launching LoginTest". invalid version "1.8.0_461": non-numeric "0_461"

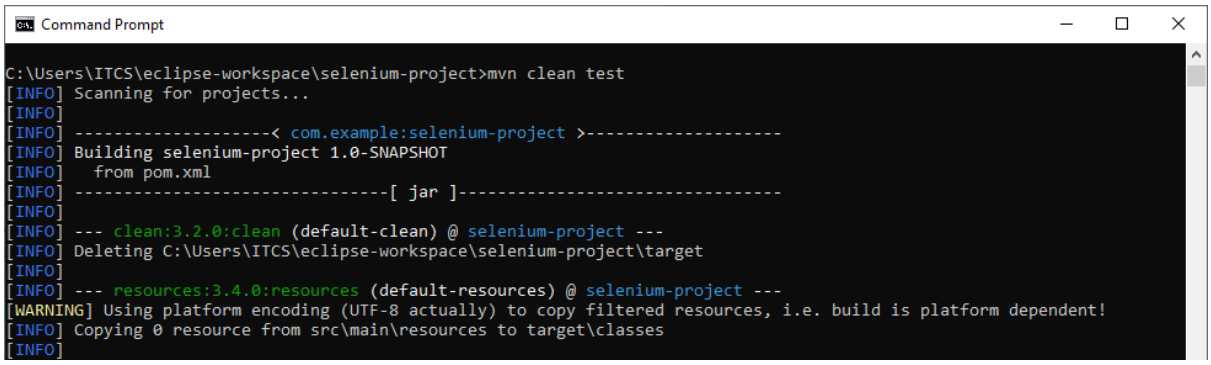
Then
**Check Eclipse's Java version**
Go to:
**Window → Preferences → Java → Installed JREs**
You may see something like:
jre1.8.0_461
You need to add/select your **JDK 21** instead.

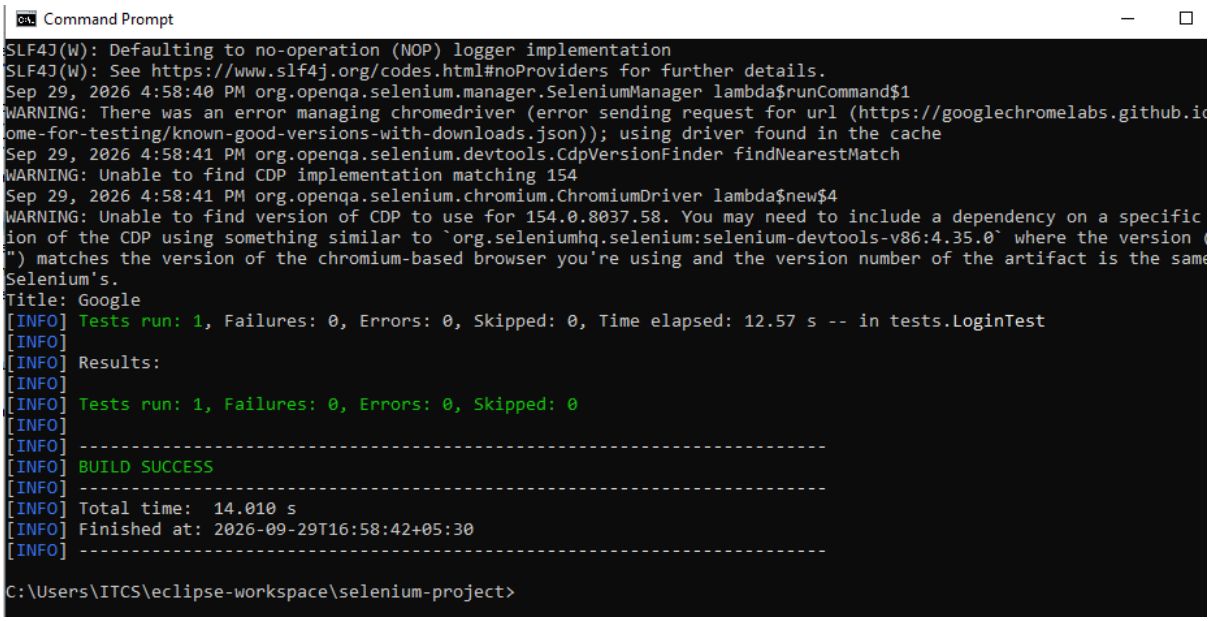
**Check the project's JRE**
Right-click:
**selenium-project → Properties → Java Build Path → Libraries**
Look for:
JRE System Library [JavaSE-1.8]
If it says Java 8, remove it and add:
JRE System Library → Workspace default JRE
or select:
JDK 21
Click **Apply and Close**.

**3. Check Java Compiler**
Go to:
**Project → Properties → Java Compiler**
Set:
Compiler compliance level: 21
Click **Apply and Close**.

**Step 14 : Run Through Maven**
This is important because **Jenkins will use Maven to execute your Selenium project**.
Open Command Prompt.

Go to your Eclipse workspace/project.
For example: **cd eclipse-workspace\selenium-project**
Run: **mvn clean test**
You should eventually see:





If you get BUILD SUCCESS, you're ready for Jenkins.

If getting BUILD FAILURE then type following steps -
C:\Users\ITCS>**set JAVA_HOME=C:\Program Files\Java\jdk-21.0.12**
C:\Users\ITCS >**set PATH=%JAVA_HOME%\bin;%PATH%**
C:\Users\ITCS>**java -version**

O/P - java version "21.0.12" 2026-07-21 LTS
Java(TM) SE Runtime Environment (build 21.0.12+7-LTS-205)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.12+7-LTS-205, mixed mode, sharing)

C:\Users\ITCS>**javac -version**
javac 21.0.12

C:\Users\ITCS>**mvn -version**
O/P - Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)
Maven home: D:\STQA\apache-maven-3.9.16
Java version: 21.0.12, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-21.0.12
Default locale: en_IN, platform encoding: UTF-8
OS name: "windows 10", version: "10.0", arch: "amd64", family: "windows"

C:\Users\ITCS>**cd eclipse-workspace\selenium-project**

C:\Users\ITCS\eclipse-workspace\selenium-project>**mvn clean test**

**Step 15 - Check the Generated Report**
After running: mvn clean test
Maven creates: target

Your project now contains:
selenium-project

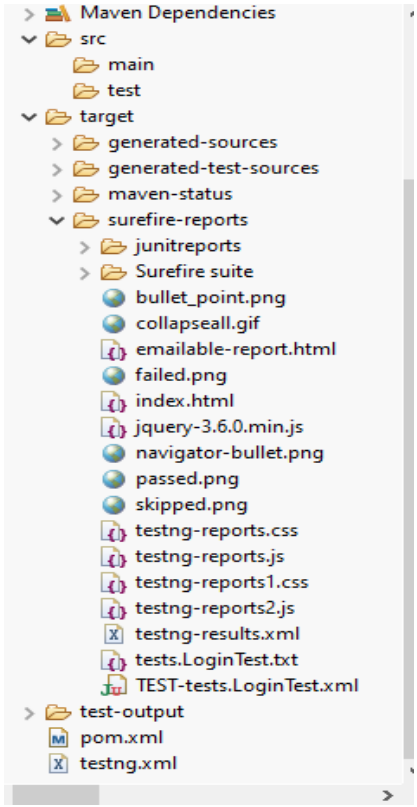
```
├── pom.xml
├── testng.xml
```

```
├── src
│   └── test
│       └── java
│           └── tests
│               └── LoginTest.java
└── target
    └── surefire-reports
```

Inside:
target/surefire-reports
you'll find XML test-result files.

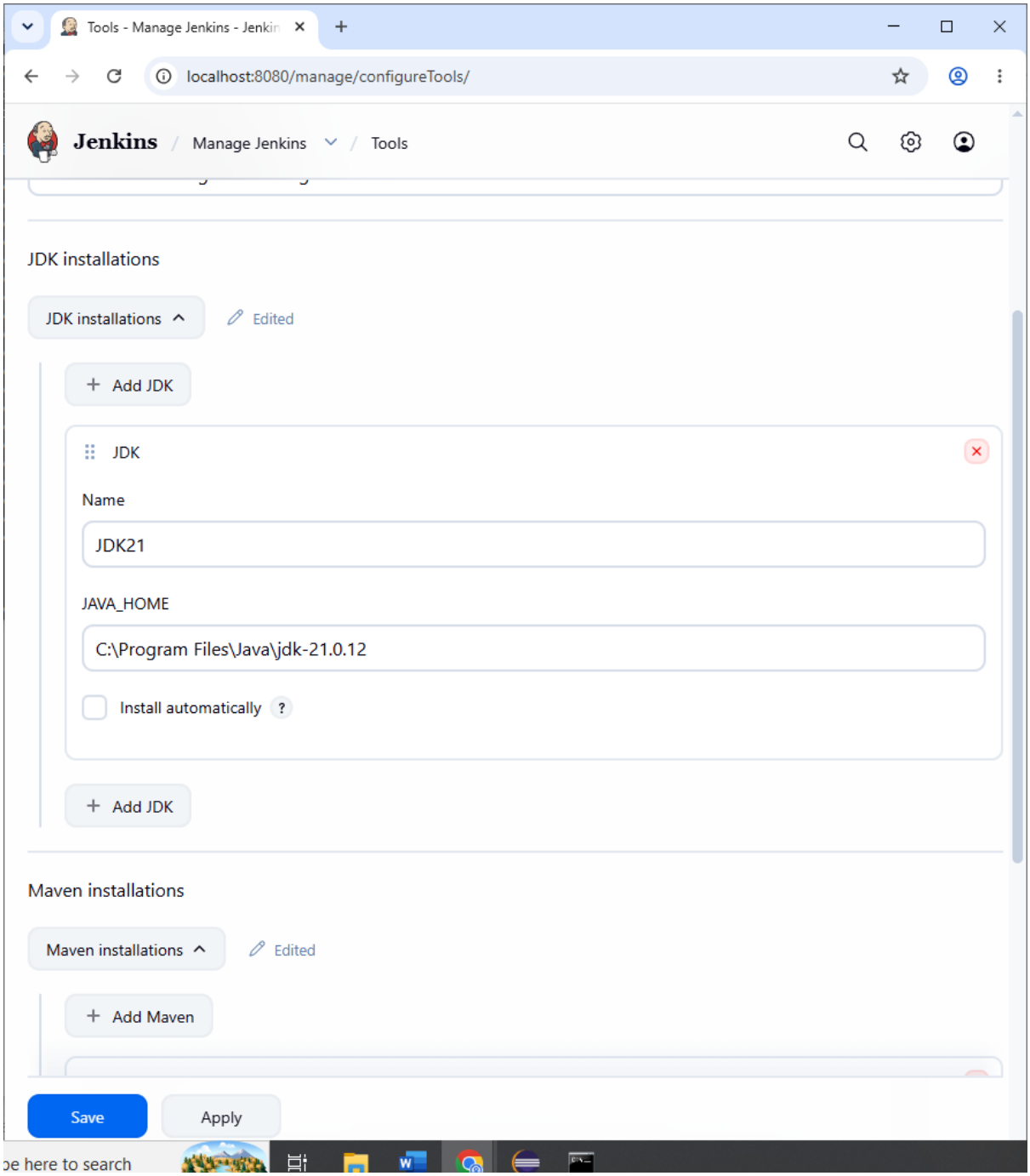
**These files will later be read by Jenkins to generate the test report.**

**Step 16 — Refresh Eclipse**
Go back to Eclipse.
In the left Project Explorer:
selenium-project
Right-click it → **Refresh**
or press:
F5
Then expand:

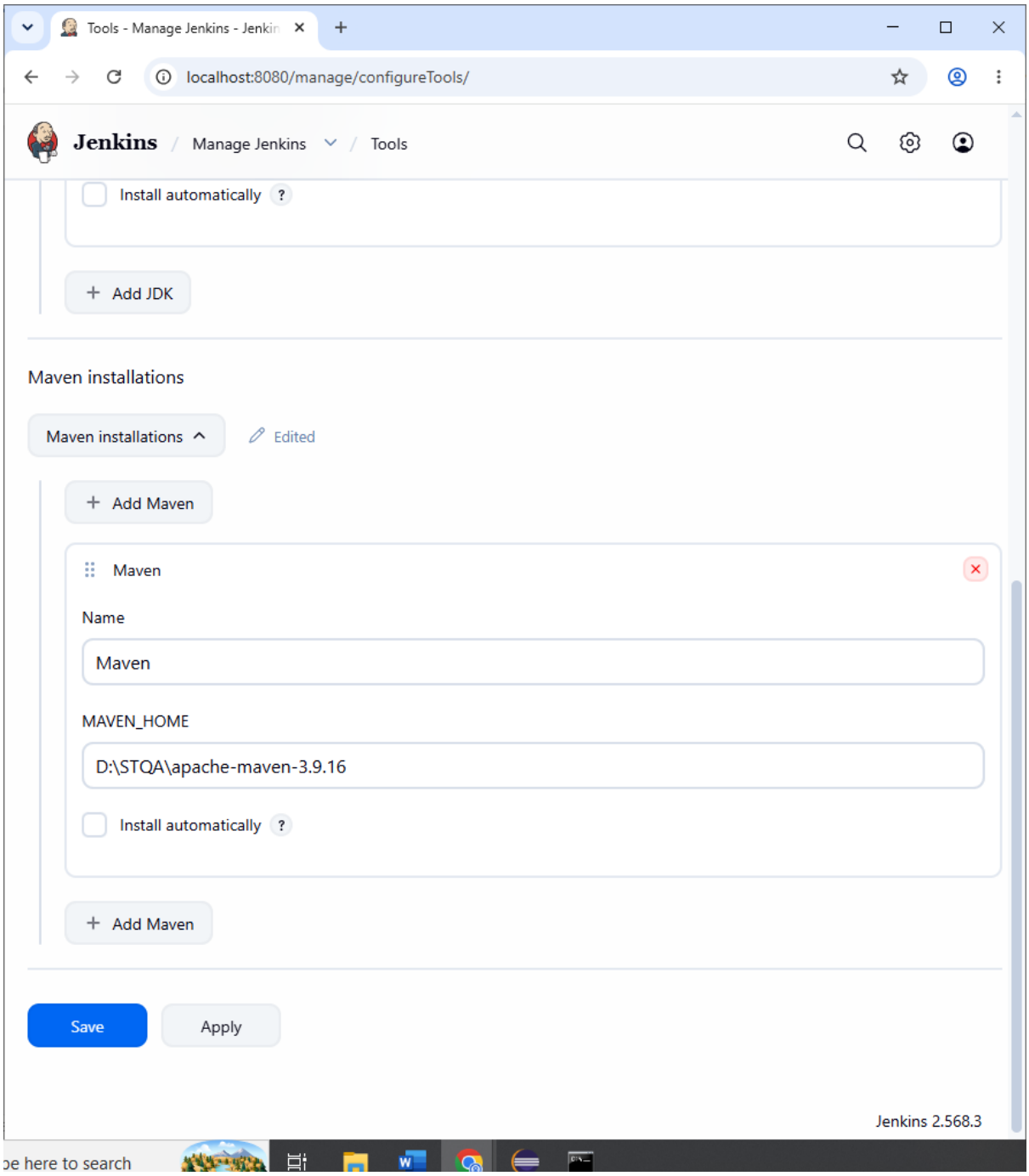



**Step 17 — Open Jenkins**
Open your browser:
http://localhost:8080
Log in.

**Step 18 — Configure JDK and Maven**
Go to settings -
**Manage Jenkins → Tools**
Configure your JDK.
For example:
JDK installation name: JDK21 and provide the path in java_home




Configure Maven: Maven installation name:
Maven
Save.



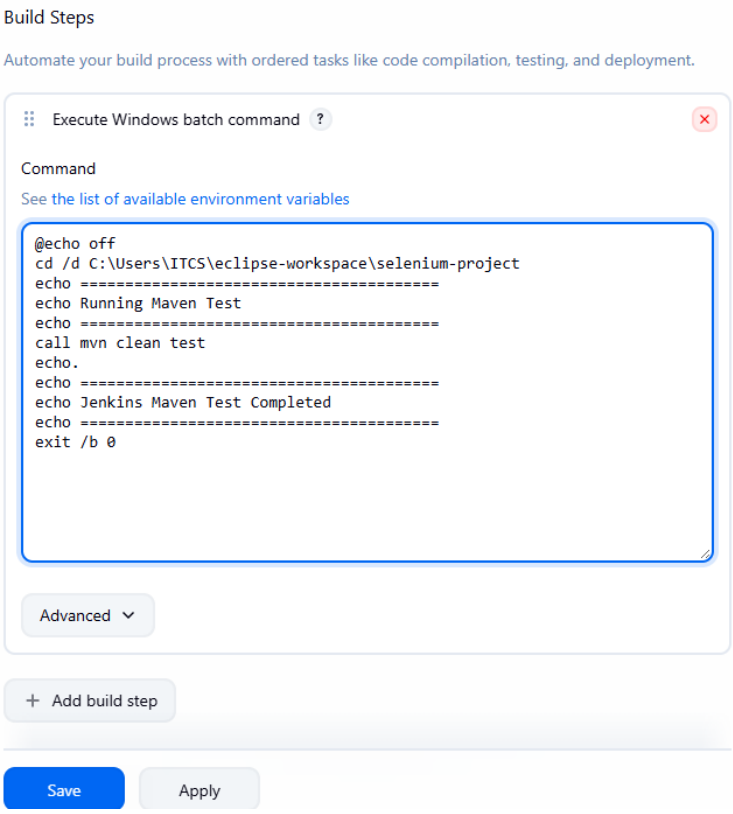

**Step 19 — Create Jenkins Job**
From the Jenkins Dashboard:
Click:
**New Item**
Enter:
Selenium-CI
Select:
**Freestyle project**
Click:
**OK**

**Step 20 — Add the build command**
Scroll down to:

**Build Steps**
Click:
**Add build step → Execute Windows batch command**
Enter:

```
@echo off
cd /d C:\Users\ITCS\eclipse-workspace\selenium-project
echo ========================================
echo Running Maven Test
echo ========================================
call mvn clean test
echo.
echo ========================================
echo Jenkins Maven Test Completed
echo ========================================
exit /b 0
```

So Jenkins will execute your exact project.




Click: **Save**
Then: **Build Now**

**Step 21 – Expected output -**
Jenkins should execute:

```
http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException

Maven completed with exit code: 1

========================================
Jenkins build marked SUCCESS
========================================
Finished: SUCCESS
```

